

RTL with Verilog Lab - Week #4

VGA Controller with SRAM Interface

Objective

The primary objective of this lab is to design and implement a VGA (Video Graphics Array) Controller that interfaces with an external SRAM (Static Random Access Memory) to display images on a VGA monitor. This lab focuses on developing a complete video display system using RTL (Register Transfer Level) design methodology in Verilog.

Design Code:

```
module VGA_Controller(  
    input wire clk,          // 50 MHz system clock  
    input wire reset,        // Active high reset  
    input wire [15:0] data_in, // Data from SRAM  
    output wire h_sync,      // Horizontal sync  
    output wire v_sync,      // Vertical sync  
    output wire [3:0] red,    // Red color output (4-bit)  
    output wire [3:0] green,  // Green color output (4-bit)  
    output wire [3:0] blue,   // Blue color output (4-bit)  
    output wire [18:0] sram_addr, // SRAM address  
    output wire sram_ce,      // SRAM chip enable  
    output wire sram_oe,      // SRAM output enable  
    output wire sram_we       // SRAM write enable  
);  
  
    // VGA Timing Parameters  
    parameter H_FRONT = 16;  
    parameter H_SYNC  = 96;  
    parameter H_BACK  = 48;  
    parameter H_ACTIVE = 640;
```

```
parameter H_TOTAL = 800;
```

```
parameter V_FRONT = 11;
```

```
parameter V_SYNC = 2;
```

```
parameter V_BACK = 31;
```

```
parameter V_ACTIVE = 480;
```

```
parameter V_TOTAL = 525;
```

```
// Internal signals
```

```
reg [9:0] h_count;    // Horizontal counter
```

```
reg [9:0] v_count;    // Vertical counter
```

```
reg [18:0] addr_reg;  // Address register
```

```
reg [15:0] pixel_data; // Pixel data from SRAM
```

```
reg h_sync_reg, v_sync_reg;
```

```
reg video_on;
```

```
// Pixel position
```

```
wire [9:0] pixel_x = h_count - (H_FRONT + H_SYNC + H_BACK);
```

```
wire [9:0] pixel_y = v_count - (V_FRONT + V_SYNC + V_BACK);
```

```
// SRAM control signals
```

```
assign sram_ce = 1'b0; // Always enabled
```

```
assign sram_oe = 1'b0; // Always output enabled
```

```
assign sram_we = 1'b1; // Read only
```

```
// Generate SRAM address from pixel position
```

```
assign sram_addr = (pixel_y * 640) + pixel_x;
```

```
// Horizontal counter
```

```
always @(posedge clk or posedge reset) begin
```

```

    if (reset)
        h_count <= 0;
    else begin
        if (h_count == H_TOTAL - 1)
            h_count <= 0;
        else
            h_count <= h_count + 1;
        end
    end
end

// Vertical counter
always @(posedge clk or posedge reset) begin
    if (reset)
        v_count <= 0;
    else begin
        if (h_count == H_TOTAL - 1) begin
            if (v_count == V_TOTAL - 1)
                v_count <= 0;
            else
                v_count <= v_count + 1;
            end
        end
    end
end

// Horizontal sync generation
always @(posedge clk or posedge reset) begin
    if (reset)
        h_sync_reg <= 1'b1;
    else begin
        if (h_count >= H_FRONT && h_count < H_FRONT + H_SYNC)

```

```

        h_sync_reg <= 1'b0;
    else
        h_sync_reg <= 1'b1;
    end
end

// Vertical sync generation
always @(posedge clk or posedge reset) begin
    if (reset)
        v_sync_reg <= 1'b1;
    else begin
        if (v_count >= V_FRONT && v_count < V_FRONT + V_SYNC)
            v_sync_reg <= 1'b0;
        else
            v_sync_reg <= 1'b1;
        end
    end
end

// Video on signal
always @(posedge clk or posedge reset) begin
    if (reset)
        video_on <= 1'b0;
    else begin
        if (h_count >= H_FRONT + H_SYNC + H_BACK &&
            h_count < H_FRONT + H_SYNC + H_BACK + H_ACTIVE &&
            v_count >= V_FRONT + V_SYNC + V_BACK &&
            v_count < V_FRONT + V_SYNC + V_BACK + V_ACTIVE)
            video_on <= 1'b1;
        else
            video_on <= 1'b0;
        end
    end
end

```

```

        end

end

// Pixel data output (with color mapping)
assign red = video_on ? pixel_data[15:12] : 4'b0000;
assign green = video_on ? pixel_data[11:8] : 4'b0000;
assign blue = video_on ? pixel_data[7:4] : 4'b0000;

// Assign sync outputs
assign h_sync = h_sync_reg;
assign v_sync = v_sync_reg;

endmodule

Test bench:

`timescale 1ns / 1ps

module VGA_Controller_TB;

    // Inputs
    reg clk;
    reg reset;
    reg [15:0] data_in;

    // Outputs
    wire h_sync;
    wire v_sync;
    wire [3:0] red;
    wire [3:0] green;
    wire [3:0] blue;
    wire [18:0] sram_addr;

```

```

wire sram_ce;

wire sram_oe;

wire sram_we;


// Instantiate the Unit Under Test (UUT)
VGA_Controller uut (
    .clk(clk),
    .reset(reset),
    .data_in(data_in),
    .h_sync(h_sync),
    .v_sync(v_sync),
    .red(red),
    .green(green),
    .blue(blue),
    .sram_addr(sram_addr),
    .sram_ce(sram_ce),
    .sram_oe(sram_oe),
    .sram_we(sram_we)
);


// Clock generation (50 MHz -> 20ns period)
initial begin
    clk = 0;
    forever #10 clk = ~clk;
end


// Stimulus
initial begin
    // Initialize inputs
    reset = 1;

```

```
data_in = 16'hFFFF; // White color

// Apply reset
#40;
reset = 0;

// Wait for a few frames
#4000000; // Simulate for 4ms (approx 4 frames at 60Hz)

// Change data
data_in = 16'hF000; // Red

#2000000; // Simulate for 2ms

data_in = 16'h0F00; // Green

#2000000; // Simulate for 2ms

data_in = 16'h00F0; // Blue

#2000000; // Simulate for 2ms

// Test pattern - checkerboard
data_in = 16'hFFFF;

#2000000;

$stop;

end
```

```

// Monitor output
initial begin
    $monitor("Time: %t, h_sync: %b, v_sync: %b, red: %h, green: %h, blue: %h, addr:
%h",
        $time, h_sync, v_sync, red, green, blue, sram_addr);
end

// VGA timing verification
reg [9:0] h_count_tb = 0;
reg [9:0] v_count_tb = 0;

always @(posedge clk) begin
    if (reset) begin
        h_count_tb <= 0;
        v_count_tb <= 0;
    end else begin
        if (h_count_tb == 800 - 1) begin
            h_count_tb <= 0;
            if (v_count_tb == 525 - 1)
                v_count_tb <= 0;
            else
                v_count_tb <= v_count_tb + 1;
        end else begin
            h_count_tb <= h_count_tb + 1;
        end
    end
end

// Check for proper sync signals
always @(posedge clk) begin
    // Check horizontal sync timing

```

```

if (h_count_tb >= 16 && h_count_tb < 112) begin
    if (h_sync !== 1'b0)
        $display("ERROR: h_sync should be 0 at h_count = %d", h_count_tb);
end else begin
    if (h_sync !== 1'b1)
        $display("ERROR: h_sync should be 1 at h_count = %d", h_count_tb);
end

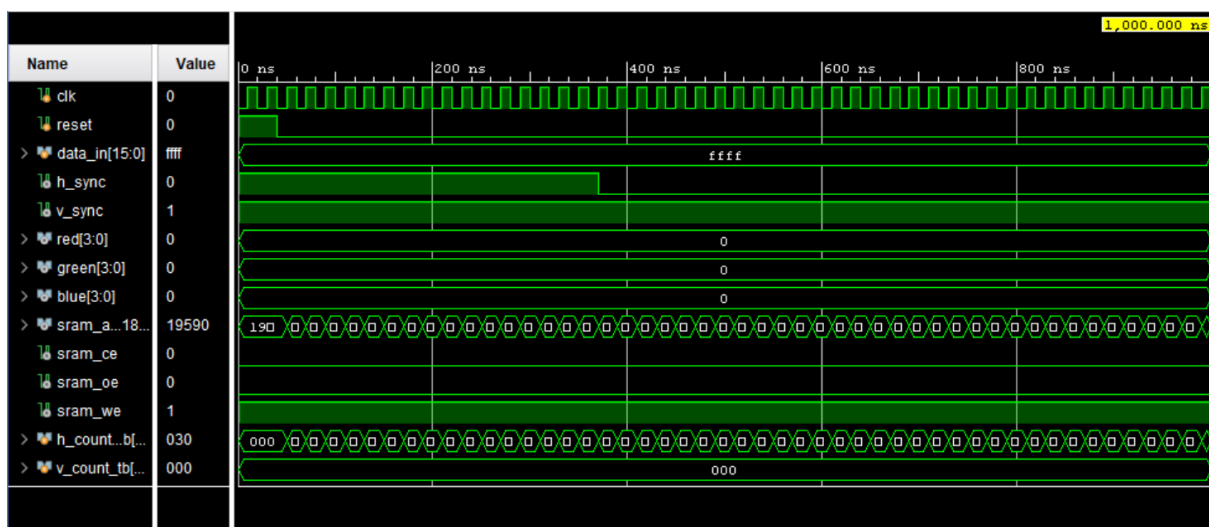
// Check vertical sync timing
if (v_count_tb >= 11 && v_count_tb < 13) begin
    if (v_sync !== 1'b0)
        $display("ERROR: v_sync should be 0 at v_count = %d", v_count_tb);
end else begin
    if (v_sync !== 1'b1)
        $display("ERROR: v_sync should be 1 at v_count = %d", v_count_tb);
end

end

endmodule

```

Wave Form:



Synthesis:

